Отчет по практической работе №2

Сравнительный анализ моделей доступа и проектирование RBAC-модели

Часть 1

1.1 Анализ сценариев

Сценарий A. Стартап с 5 сотрудниками

1. Для небольшой команды подойдет модель DAC, потому что сотрудников мало и права доступа можно выдавать каждому отдельно.
2. Плюсы этой модели — простота и удобство. Минус — можно случайно дать кому-то лишние права.
3. Когда сотрудников станет больше, лучше использовать RBAC, чтобы распределять права по ролям.

Сценарий B. Военное ведомство

1. Для военного ведомства подойдет модель MAC, потому что информация имеет разные уровни секретности.
2. No Read Up означает, что нельзя читать документы выше своего уровня допуска. No Write Down означает, что нельзя записывать информацию в документы с более низким уровнем секретности.
3. Для дополнительной защиты нужны надежные пароли, шифрование данных, запись действий пользователей и резервные копии.

Сценарий C. Корпорация с 500 сотрудниками

1. Для большой компании подойдет модель RBAC, потому что в ней удобно распределять права между сотрудниками по ролям.
2. Можно создать роли сотрудников, менеджеров и директоров. У каждой роли будут свои права доступа.
3. Для разделения обязанностей (SoD) можно сделать так, чтобы один сотрудник создавал платеж, а другой его подтверждал.

1.2 Сравнительная таблица моделей доступа

Критерий	**DAC**	**MAC**	**RBAC**
Простота внедрения	5/5	2/5	4/5
Масштабируемость	2/5	3/5	5/5
Гибкость	5/5	2/5	4/5
Уровень безопасности	3/5	5/5	4/5
Пригодность для аудита	2/5	5/5	5/5
Стоимость поддержки	4/5	1/5	4/5

Вывод: DAC больше подходит для небольших организаций, потому что она простая и гибкая. MAC лучше использовать там, где нужна строгая защита информации. RBAC удобнее для больших компаний, так как права распределяются по ролям.

Часть 2

2.1 Создание схемы базы данных

практика 2.sql

C: > Users > Genk0s > Desktop > практика 2.sql

localhost | task_management | schema: session default

```sql
-- Таблица пользователей
CREATE TABLE app.users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица проектов
CREATE TABLE app.projects (
    project_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    owner_id INTEGER REFERENCES app.users(user_id),
    status VARCHAR(20) DEFAULT 'active'
        CHECK (status IN ('active', 'completed', 'archived')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица задач
CREATE TABLE app.tasks (
    task_id SERIAL PRIMARY KEY,
    project_id INTEGER REFERENCES app.projects(project_id),
    title VARCHAR(200) NOT NULL,
    description TEXT,
    status VARCHAR(20) DEFAULT 'todo'
        CHECK (status IN ('todo', 'in_progress', 'done')),
    priority INTEGER DEFAULT 3 CHECK (priority BETWEEN 1 AND 5),
    assignee_id INTEGER REFERENCES app.users(user_id),
    created_by INTEGER REFERENCES app.users(user_id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- Таблица комментариев к задачам
CREATE TABLE app.comments (
    comment_id SERIAL PRIMARY KEY,
    task_id INTEGER REFERENCES app.tasks(task_id),
    user_id INTEGER REFERENCES app.users(user_id),
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица истории изменений задач
CREATE TABLE app.task_history (
    history_id SERIAL PRIMARY KEY,
    task_id INTEGER REFERENCES app.tasks(task_id),
    changed_by INTEGER REFERENCES app.users(user_id),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    field_name VARCHAR(50) NOT NULL,
    old_value TEXT,
    new_value TEXT
);

-- Таблица логов доступа
CREATE TABLE app.access_logs (
    log_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES app.users(user_id),
    action VARCHAR(50) NOT NULL,
    resource_type VARCHAR(50),
    resource_id INTEGER,
    ip_address INET,
    logged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- Добавление пользователей
INSERT INTO app.users (username, email, password_hash, full_name) VALUES
('alice', 'alice@company.com', 'hash1', 'Alice Johnson'),
('bob', 'bob@company.com', 'hash2', 'Bob Smith'),
('charlie', 'charlie@company.com', 'hash3', 'Charlie Brown'),
('diana', 'diana@company.com', 'hash4', 'Diana Prince'),
('eve', 'eve@company.com', 'hash5', 'Eve Wilson');

-- Добавление проектов
INSERT INTO app.projects (name, description, owner_id, status) VALUES
('Website Redesign', 'Redesign company website', 1, 'active'),
('Mobile App', 'Develop mobile application', 2, 'active');

-- Добавление задач
INSERT INTO app.tasks
(project_id, title, description, status, priority, assignee_id, created_by)
VALUES
(1, 'Design homepage', 'Create new homepage design', 'in_progress', 1, 1, 1),
(1, 'Implement navigation', 'Add responsive navigation', 'todo', 2, 2, 1),
(2, 'Setup project', 'Initialize project', 'done', 1, 3, 2),
(2, 'Create login screen', 'Create login page', 'in_progress', 2, 4, 2),
(2, 'API integration', 'Connect to backend API', 'todo', 3, 3, 2),
(1, 'Prepare logo', 'Create company logo', 'todo', 3, 5, 1),
(1, 'Check mobile layout', 'Test mobile version', 'todo', 2, 2, 1),
(1, 'Update footer', 'Change website footer', 'done', 4, 3, 1),
(2, 'Test login', 'Check login function', 'todo', 2, 5, 2),
(2, 'Write documentation', 'Describe project', 'todo', 3, 4, 2);
```

```sql
-- Проверка пользователей
SELECT * FROM app.users;

-- Проверка проектов
SELECT * FROM app.projects;

-- Проверка задач
SELECT * FROM app.tasks;
```

Results Messages localho... | task_managem... | schema: session default

	user_id	username	email	password_hash	full_name	is_active	created_at
1	1	alice	alice@company.com	hash1	Alice Johnson	True	2026-11-02 19:54:37.161082
2	2	bob	bob@company.com	hash2	Bob Smith	True	2026-11-02 19:54:37.161082
3	3	charlie	charlie@company.com	hash3	Charlie Brown	True	2026-11-02 19:54:37.161082
4	4	diana	diana@company.com	hash4	Diana Prince	True	2026-11-02 19:54:37.161082
5	5	eve	eve@company.com	hash5	Eve Wilson	True	2026-11-02 19:54:37.161082

	project_id	name	description	owner_id	status	created_at
1	1	Website Redesign	Redesign company website	1	active	2026-11-02 19:54:37.164592
2	2	Mobile App	Develop mobile application	2	active	2026-11-02 19:54:37.164592

	task_id	project_id	title	description	status	priority	assignee_id	created_by	created_at	updated_at
1	1	1	Design homepage	Create new homepage design	in_progress	1	1	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
2	2	1	Implement navigation	Add responsive navigation	todo	2	2	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
3	3	2	Setup project	Initialize project	done	1	3	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
4	4	2	Create login screen	Create login page	in_progress	2	4	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
5	5	2	API integration	Connect to backend API	todo	3	3	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
6	6	1	Prepare logo	Create company logo	todo	3	5	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
7	7	1	Check mobile layout	Test mobile version	todo	2	2	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
8	8	1	Update footer	Change website footer	done	4	3	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
9	9	2	Test login	Check login function	todo	2	5	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
10	10	2	Write documentation	Describe project	todo	3	4	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512

В PostgreSQL была создана база данных task_management.

В схеме были созданы 6 таблиц: users, projects, tasks, comments, task_history и access_logs.

После создания таблиц были добавлены тестовые данные: 5 пользователей, 2 проекта и 10 задач.

С помощью запросов SELECT была выполнена проверка добавленных данных.

2.2. Проектирование ролей

У нас будет 5 ролей:

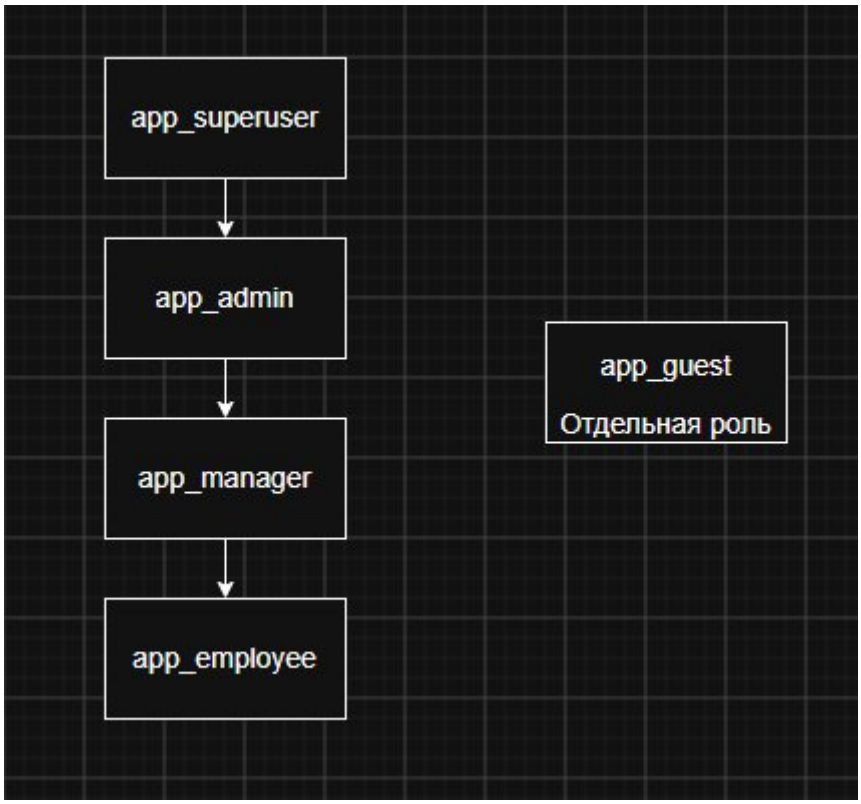
app_guest — гость.

app_employee — сотрудник.

app_manager — менеджер.

app_admin — администратор.

app_superuser — роль с самыми широкими правами.



Верхняя роль получает права нижней. Например, менеджер имеет права сотрудника и дополнительные возможности. А гость находится отдельно.

Матрица привилегий

Объект / Операция	**guest**	**employee**	**manager**	**admin**	**superuser**
users — SELECT	—	Свои данные	Все	Все	Все
users — INSERT/UPDATE	—	—	—	✓	✓
projects — SELECT	Публичные	Все	Все	Все	Все
projects — INSERT/UPDATE/DELETE	—	—	Свои проекты	Все	Все
tasks — SELECT	—	Свои + общие	Все	Все	Все
tasks — INSERT	—	Свои	Любые	Все	Все
tasks — UPDATE	—	Свои (статус)	Любые	Все	Все
tasks — DELETE	—	—	Свои проекты	Все	Все
comments — SELECT/INSERT	—	✓	✓	✓	✓
task_history — SELECT	—	—	Свои проекты	Все	Все
access_logs — SELECT	—	—	—	✓	✓
DDL (CREATE/ALTER/DROP)	—	—	—	В своей схеме	Все

2.3 Реализация ролей в PostgreSQL

```
-- Шаг 1. Создание ролей
CREATE ROLE app_guest;
CREATE ROLE app_employee;
CREATE ROLE app_manager;
CREATE ROLE app_admin;
CREATE ROLE app_superuser;

-- Шаг 2. Права на подключение и использование схемы
GRANT CONNECT ON DATABASE task_management TO app_guest;
GRANT USAGE ON SCHEMA app TO app_guest;

GRANT CONNECT ON DATABASE task_management TO app_employee;
GRANT USAGE ON SCHEMA app TO app_employee;

-- Шаг 3. Права гостя
GRANT SELECT ON TABLE app.projects TO app_guest;

-- Шаг 4. Права сотрудника
GRANT SELECT (user_id, username, full_name)
ON TABLE app.users TO app_employee;

GRANT SELECT ON TABLE app.projects TO app_employee;
GRANT SELECT ON TABLE app.tasks TO app_employee;

GRANT SELECT, INSERT ON TABLE app.tasks TO app_employee;
GRANT USAGE ON SEQUENCE app.tasks_task_id_seq TO app_employee;

GRANT SELECT, INSERT ON TABLE app.comments TO app_employee;
GRANT USAGE ON SEQUENCE app.comments_comment_id_seq TO app_employee;

GRANT SELECT ON TABLE app.task_history TO app_employee;

-- Шаг 5. Права менеджера
GRANT app_employee TO app_manager;

GRANT SELECT, INSERT, UPDATE
ON TABLE app.projects TO app_manager;

GRANT SELECT, INSERT, UPDATE, DELETE
ON TABLE app.tasks TO app_manager;

GRANT SELECT, INSERT, DELETE
ON TABLE app.comments TO app_manager;

GRANT SELECT ON TABLE app.task_history TO app_manager;
GRANT INSERT ON TABLE app.task_history TO app_manager;
```

```sql
-- Шаг 6. Права администратора
GRANT app_manager TO app_admin;

GRANT SELECT, INSERT, UPDATE ON TABLE app.users TO app_admin;

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA app TO app_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA app TO app_admin;

GRANT CREATE ON SCHEMA app TO app_admin;

-- Шаг 7. Права роли superuser
GRANT app_admin TO app_superuser;

GRANT ALL PRIVILEGES ON DATABASE task_management TO app_superuser;

-- Шаг 8. Создание пользователей
CREATE USER alice WITH PASSWORD 'AliceSecure123!';
CREATE USER bob WITH PASSWORD 'BobSecure456!';
CREATE USER charlie WITH PASSWORD 'CharlieSecure789!';
CREATE USER diana WITH PASSWORD 'DianaSecure012!';
CREATE USER eve WITH PASSWORD 'EveSecure345!';

-- Назначение ролей пользователям
GRANT app_manager TO alice;

GRANT app_employee TO bob;
GRANT app_employee TO charlie;

GRANT app_admin TO diana;

GRANT app_superuser TO eve;
```

```sql
-- Шаг 9. Проверка созданных ролей
SELECT rolname
FROM pg_roles
WHERE rolname IN (
    'app_guest',
    'app_employee',
    'app_manager',
    'app_admin',
    'app_superuser'
)
ORDER BY rolname;

-- Проверка наследования ролей
SELECT
    r.rolname AS role_name,
    m.rolname AS member_name
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
JOIN pg_roles m ON am.member = m.oid
WHERE r.rolname LIKE 'app_%'
ORDER BY role_name, member_name;

-- Проверка прав доступа
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE table_schema = 'app'
AND grantee IN (
    'app_guest',
    'app_employee',
    'app_manager',
    'app_admin',
    'app_superuser'
)
ORDER BY grantee, table_name, privilege_type;
```

Results Messages

	rolname
1	app_admin
2	app_employee
3	app_guest
4	app_manager
5	app_superuser

	role_name	member_name
1	app_admin	app_superuser
2	app_admin	diana
3	app_employee	app_manager
4	app_employee	bob
5	app_employee	charlie
6	app_manager	alice
7	app_manager	app_admin
8	app_superuser	eve

	grantee	table_name	privilege_type
1	app_admin	access_logs	DELETE
2	app_admin	access_logs	INSERT
3	app_admin	access_logs	REFERENCES
4	app_admin	access_logs	SELECT
5	app_admin	access_logs	TRIGGER
6	app_admin	access_logs	TRUNCATE
7	app_admin	access_logs	UPDATE
8	app_admin	comments	DELETE
9	app_admin	comments	INSERT

В PostgreSQL были созданы 5 ролей: app_guest, app_employee, app_manager, app_admin и app_superuser.

Для каждой роли были настроены права доступа с помощью команды GRANT.

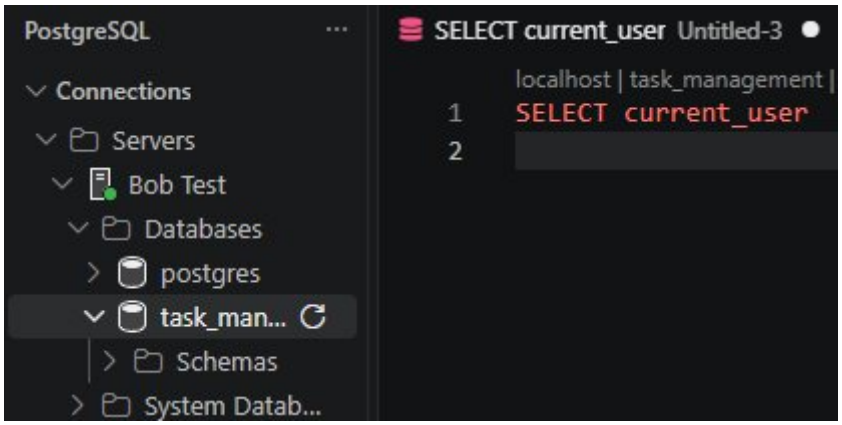
Также было настроено наследование прав. Менеджер получает права сотрудника, администратор — менеджера, а роль app_superuser — администратора.

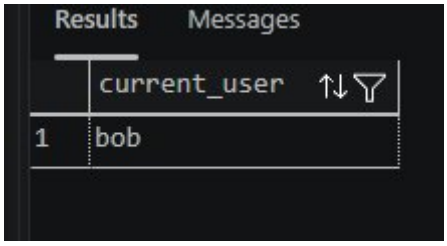
После этого были созданы пользователи alice, bob, charlie, diana и eve. Каждому пользователю назначена своя роль.

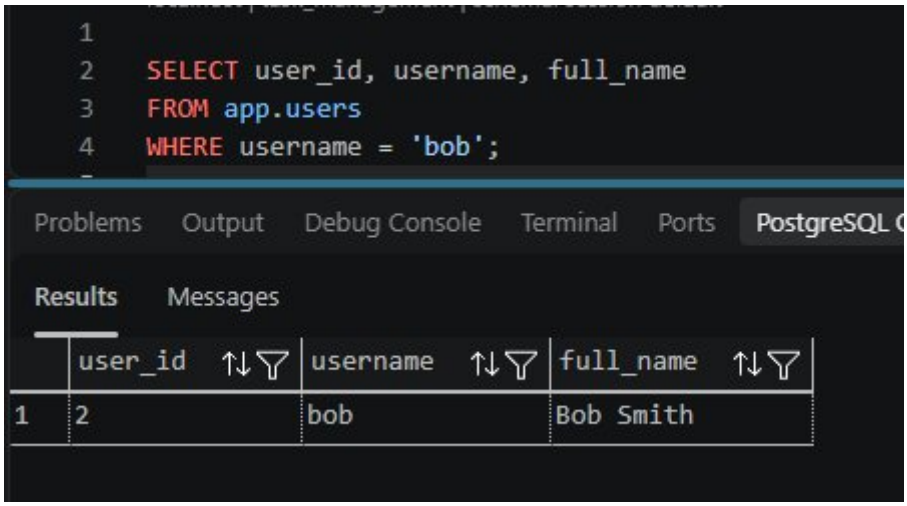
С помощью SQL-запросов была выполнена проверка созданных ролей и их прав доступа.

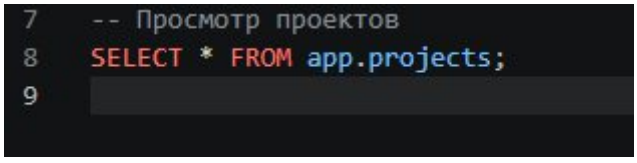
2.4 Тестирование системы доступа

BOB









	project_id	name	description	owner_id	status	created_at
1	1	Website Redesign	Redesign company website	1	active	2026-11-02 19:54:37.164592
2	2	Mobile App	Develop mobile application	2	active	2026-11-02 19:54:37.164592

```sql
-- Просмотр задач
SELECT * FROM app.tasks;
```

	task_id	project_id	title	description	status	priority	assignee_id	created_by	created_at	updated_at
1	1	1	Design homepage	Create new homepage design	in_progress	1	1	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
2	2	1	Implement navigation	Add responsive navigation	todo	2	2	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
3	3	2	Setup project	Initialize project	done	1	3	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
4	4	2	Create login screen	Create login page	in_progress	2	4	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
5	5	2	API integration	Connect to backend API	todo	3	3	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
6	6	1	Prepare logo	Create company logo	todo	3	5	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
7	7	1	Check mobile layout	Test mobile version	todo	2	2	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
8	8	1	Update footer	Change website footer	done	4	3	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
9	9	2	Test login	Check login function	todo	2	5	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
10	10	2	Write documentation	Describe project	todo	3	4	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512

```sql
-- Bob создает новую задачу
INSERT INTO app.tasks
(project_id, title, description, assignee_id, created_by)
VALUES
(1, 'Test task by Bob', 'Testing employee permissions', 2, 2);
```

	task_id	project_id	title	description	status	priority	assignee_id	created_by	created_at	updated_at
3	3	2	Setup project	Initialize project	done	1	3	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
4	4	2	Create login screen	Create login page	in_progress	2	4	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
5	5	2	API integration	Connect to backend API	todo	3	3	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
6	6	1	Prepare logo	Create company logo	todo	3	5	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
7	7	1	Check mobile layout	Test mobile version	todo	2	2	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
8	8	1	Update footer	Change website footer	done	4	3	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
9	9	2	Test login	Check login function	todo	2	5	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
10	10	2	Write documentation	Describe project	todo	3	4	2	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
11	11	1	Test task by Bob	Testing employee permissions	todo	3	2	2	2026-11-02 20:39:33.903043	2026-11-02 20:39:33.903043

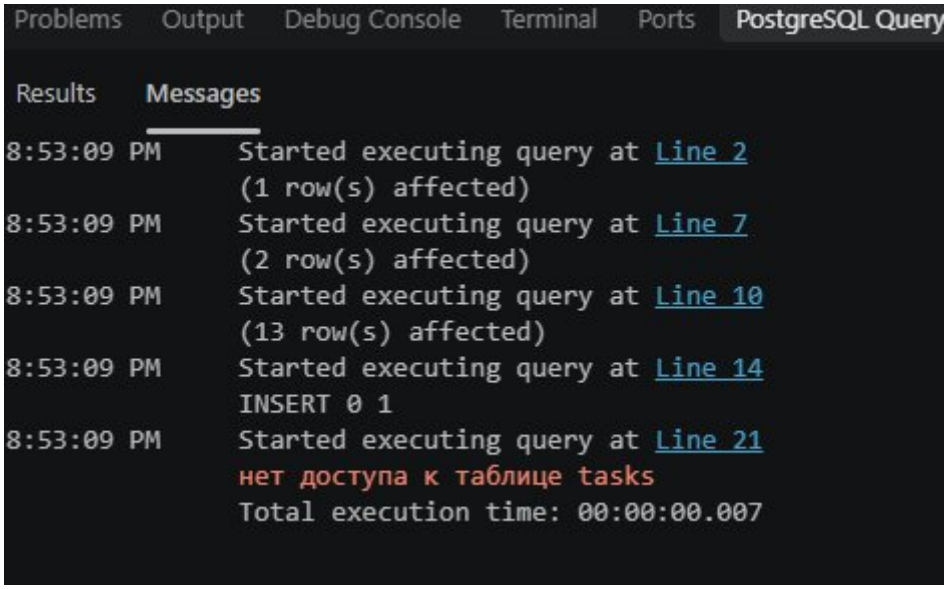
```sql
-- Bob пытается изменить задачу
UPDATE app.tasks
SET status = 'done'
WHERE task_id = 1;
```

Problems Output Debug Console Terminal Ports PostgreSQL

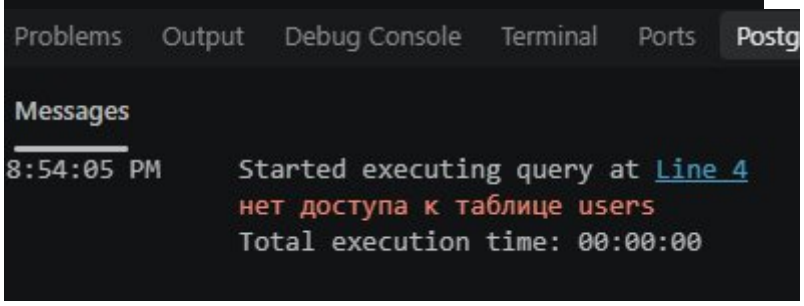
Results Messages

```
8:52:06 PM     Started executing query at Line 2
               (1 row(s) affected)
8:52:06 PM     Started executing query at Line 7
               (2 row(s) affected)
8:52:06 PM     Started executing query at Line 10
               (12 row(s) affected)
8:52:06 PM     Started executing query at Line 14
               INSERT 0 1
8:52:06 PM     Started executing query at Line 21
               нет доступа к таблице tasks
               Total execution time: 00:00:00.008
```

```
-- Bob пытается удалить задачу
DELETE FROM app.tasks
WHERE task_id = 1;
```




```
-- Bob пытается посмотреть пароли пользователей
SELECT password_hash FROM app.users;
```



Сотрудник Bob может просматривать проекты, задачи и разрешённые данные пользователей, а также создавать задачи. Изменять и удалять задачи, и просматривать пароли ему запрещено.

ALICE

```
-- Проверка текущего пользователя
SELECT current_user;
```

Problems Output Debug Console Terminal Ports

Results Messages

	current_user
1	alice

localhost | task_management | schema: session default

```
-- Alice создает задачу
INSERT INTO app.tasks (project_id, title, assignee_id, created_by)
VALUES (1, 'Task from manager', 2, 1);

-- Alice изменяет статус задачи
UPDATE app.tasks
SET status = 'in_progress'
WHERE task_id = 1;
```

Problems Output Debug Console Terminal Ports PostgreSQL Query Results

Messages

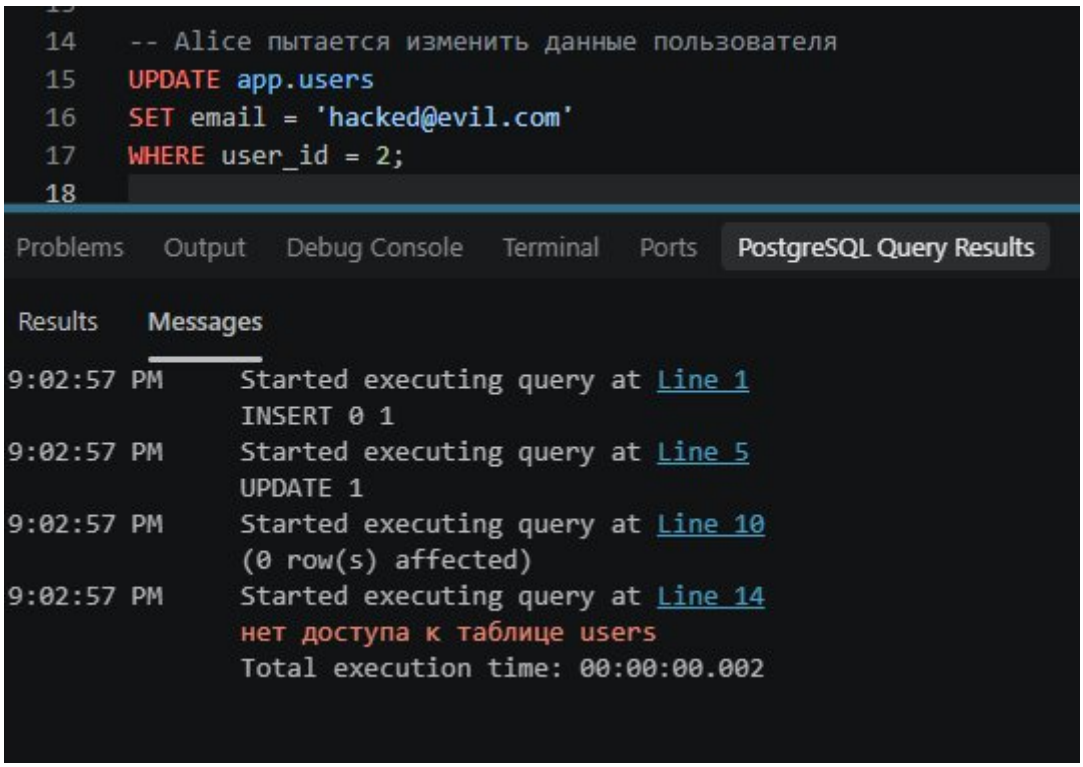
```
9:02:13 PM     Started executing query at Line 1
               INSERT 0 1
9:02:13 PM     Started executing query at Line 5
               UPDATE 1
               Total execution time: 00:00:00.002
```

```
-- Alice просматривает историю изменений
SELECT * FROM app.task_history;
```

Problems Output Debug Console Terminal Ports PostgreSQL Query Results

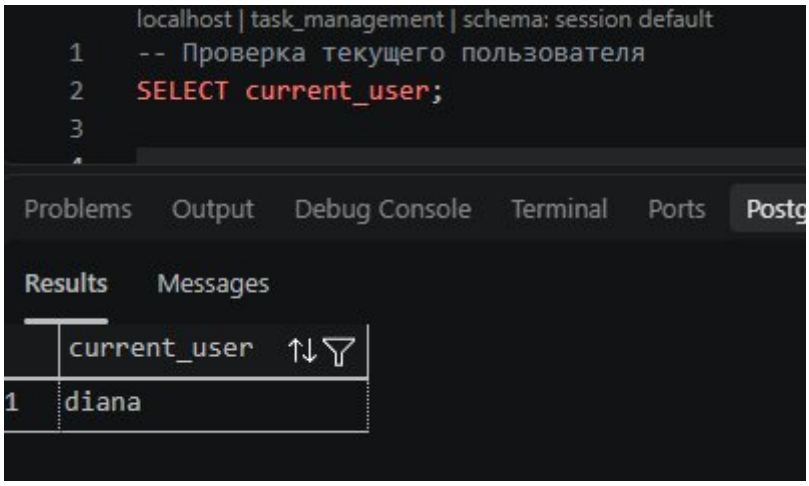
Results Messages localho... | task_managem... | schema: session default

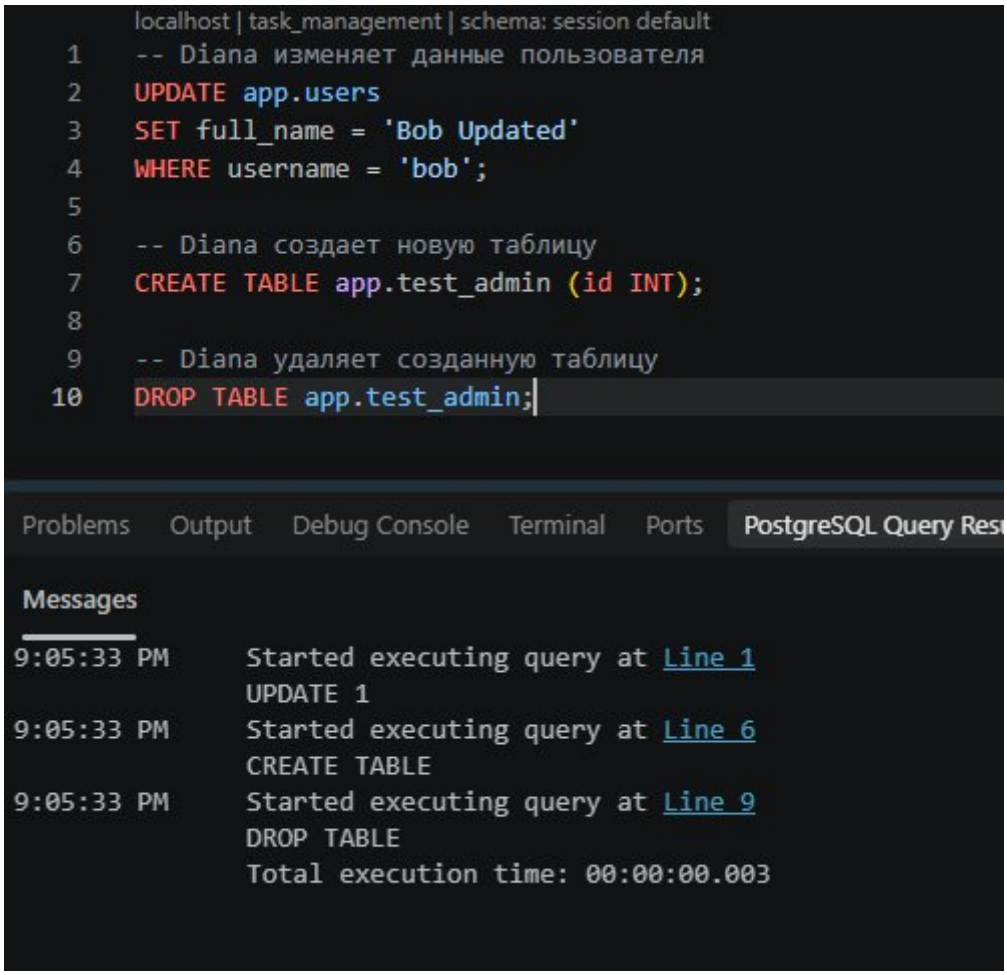
history_id	task_id	changed_by	changed_at	field_name	old_value	new_value



Менеджер Alice смог создать задачу, изменить её статус и просмотреть историю изменений. При попытке изменить данные пользователя возникла ошибка доступа.

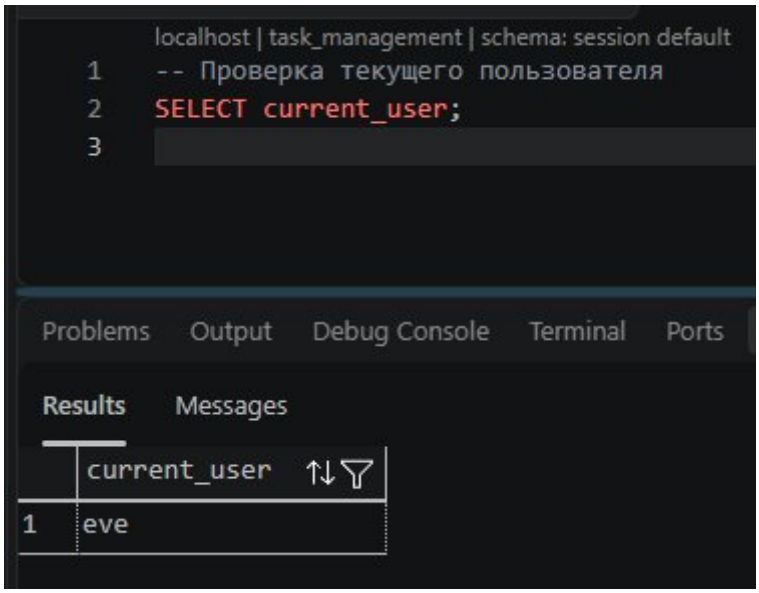
DIANA

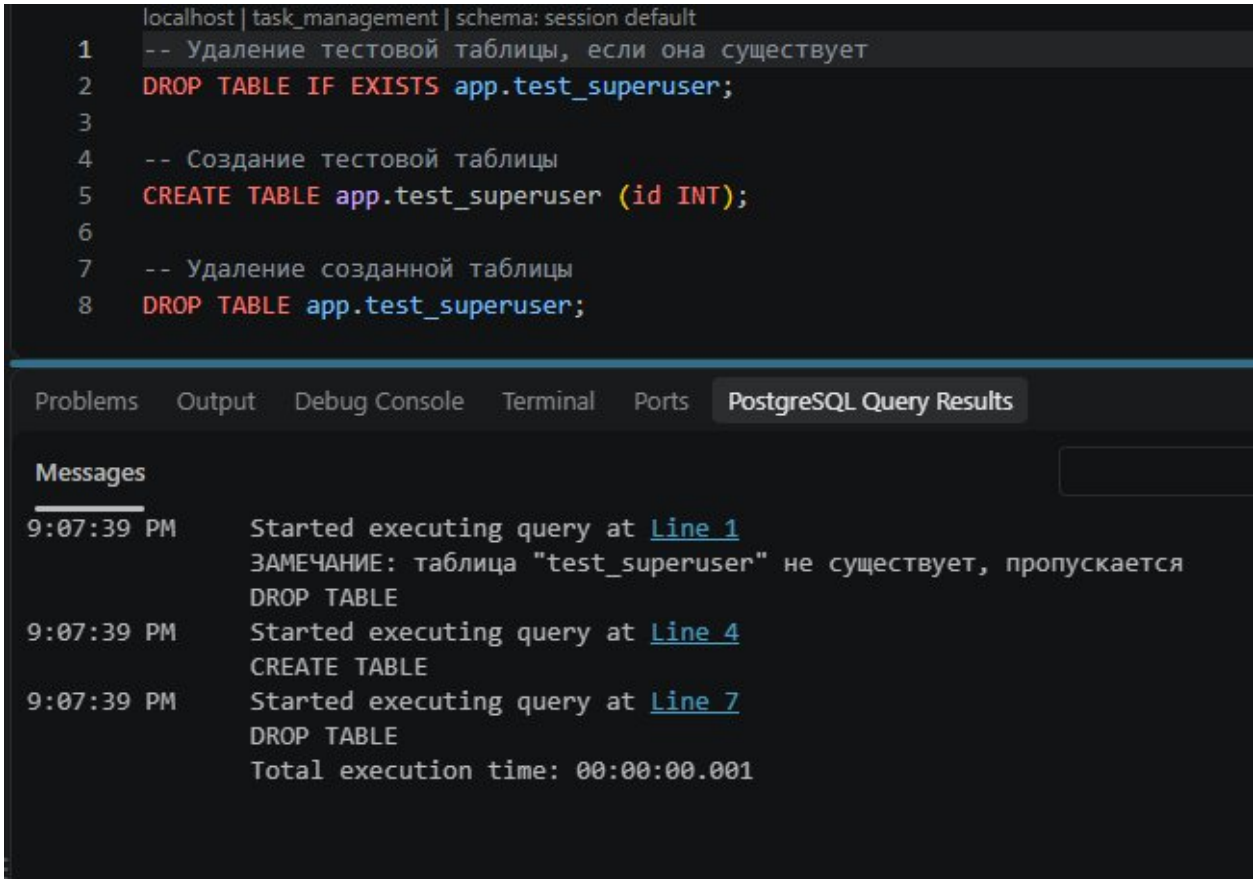




Администратор Diana смог изменить данные пользователя Bob, создать новую таблицу и удалить её.

EVE





Пользователь Eve смог создать и удалить тестовую таблицу.

2.5 Реализация ограничений на уровне строк (RLS)

```sql
ALTER TABLE app.tasks ENABLE ROW LEVEL SECURITY;

-- Сотрудники видят свои и общие задачи
CREATE POLICY employee_tasks_select ON app.tasks
FOR SELECT
TO app_employee
USING (
    assignee_id = (
        SELECT user_id FROM app.users
        WHERE username = current_user
    )
    OR assignee_id IS NULL
);

-- Сотрудники могут создавать свои и общие задачи
CREATE POLICY employee_tasks_insert ON app.tasks
FOR INSERT
TO app_employee
WITH CHECK (
    assignee_id = (
        SELECT user_id FROM app.users
        WHERE username = current_user
    )
    OR assignee_id IS NULL
);

-- Сотрудники могут обновлять только свои задачи
CREATE POLICY employee_tasks_update ON app.tasks
FOR UPDATE
TO app_employee
USING (
    assignee_id = (
        SELECT user_id FROM app.users
        WHERE username = current_user
    )
);
```

```sql
CREATE POLICY manager_tasks_all ON app.tasks
FOR ALL
TO app_manager
USING (TRUE)
WITH CHECK (TRUE);

-- Политика 3. Доступ к данным пользователей

ALTER TABLE app.users ENABLE ROW LEVEL SECURITY;

-- Сотрудники могут просматривать разрешенные данные
CREATE POLICY users_employee_select ON app.users
FOR SELECT
TO app_employee
USING (TRUE);

-- Изменение только своих данных
CREATE POLICY users_employee_update ON app.users
FOR UPDATE
TO app_employee
USING (
    username = current_user
);
```

```
Messages
9:11:05 PM     Started executing query at Line 2
               ALTER TABLE
9:11:05 PM     Started executing query at Line 8
               CREATE POLICY
9:11:05 PM     Started executing query at Line 20
               CREATE POLICY
9:11:05 PM     Started executing query at Line 32
               CREATE POLICY
9:11:05 PM     Started executing query at Line 43
               CREATE POLICY
9:11:05 PM     Started executing query at Line 51
               ALTER TABLE
9:11:05 PM     Started executing query at Line 55
               CREATE POLICY
9:11:05 PM     Started executing query at Line 61
               CREATE POLICY
               Total execution time: 00:00:00.004
```

Включили защиту RLS для таблиц tasks и users и создали правила доступа к их строкам.

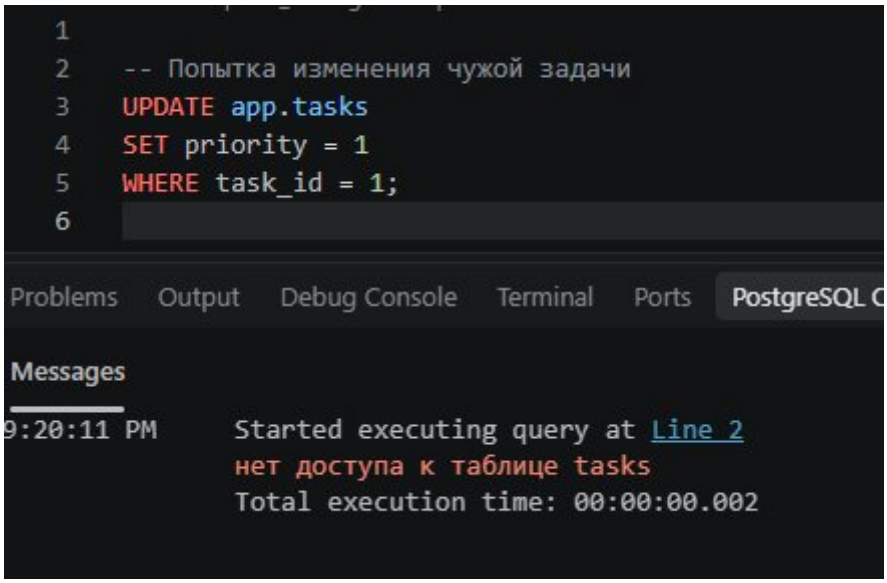
```
-- Просмотр задач после включения RLS
SELECT * FROM app.tasks;
```

Problems Output Debug Console Terminal Ports PostgreSQL Query Results

Results Messages localho... | task_managem... | schema: session default Open in New Tab

	task_id	project_id	title	description	status	priority	assignee_id	created_by	created_at	updated_at
1	2	1	Implement navigation	Add responsive navigation	todo	2	2	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
2	7	1	Check mobile layout	Test mobile version	todo	2	2	1	2026-11-02 19:54:37.166512	2026-11-02 19:54:37.166512
3	11	1	Test task by Bob	Testing employee permissions	todo	3	2	2	2026-11-02 20:39:33.903043	2026-11-02 20:39:33.903043
4	12	1	Test task by Bob	Testing employee permissions	todo	3	2	2	2026-11-02 20:50:59.311375	2026-11-02 20:50:59.311375
5	13	1	Test task by Bob	Testing employee permissions	todo	3	2	2	2026-11-02 20:52:06.952144	2026-11-02 20:52:06.952144
6	14	1	Test task by Bob	Testing employee permissions	todo	3	2	2	2026-11-02 20:53:09.63373	2026-11-02 20:53:09.63373
7	15	1	Test task by Bob	Testing employee permissions	todo	3	2	2	2026-11-02 20:53:52.44192	2026-11-02 20:53:52.44192
8	16	1	Task from manager	NULL	todo	3	2	1	2026-11-02 20:58:44.683006	2026-11-02 20:58:44.683006
9	17	1	Task from manager	NULL	todo	3	2	1	2026-11-02 20:58:53.040484	2026-11-02 20:58:53.040484
10	18	1	Task from manager	NULL	todo	3	2	1	2026-11-02 21:00:07.822071	2026-11-02 21:00:07.822071
11	19	1	Task from manager	NULL	todo	3	2	1	2026-11-02 21:00:28.475282	2026-11-02 21:00:28.475282
12	20	1	Task from manager	NULL	todo	3	2	1	2026-11-02 21:00:36.308661	2026-11-02 21:00:36.308661
13	21	1	Task from manager	NULL	todo	3	2	1	2026-11-02 21:02:13.911707	2026-11-02 21:02:13.911707
14	22	1	Task from manager	NULL	todo	3	2	1	2026-11-02 21:02:35.771394	2026-11-02 21:02:35.771394
15	23	1	Task from manager	NULL	todo	3	2	1	2026-11-02 21:02:57.621914	2026-11-02 21:02:57.621914

Раньше Bob видел все задачи. Теперь количество строк должно уменьшиться.



В PostgreSQL была настроена защита на уровне строк (RLS) для таблиц tasks и users.

Для сотрудников были созданы правила, которые позволяют просматривать свои и общие задачи, а также добавлять задачи.

Для менеджеров настроен более широкий доступ к задачам.